

pacman and paru

1. Development and distribution of Archlinux packages
2. pacman
3. paru
- 4.

Development and distribution of Archlinux packages

1. Official maintainers develop official repositories.
2. Then, official maintainers push official repositories to Git hosting server.
3. Official maintainers build packages/utilities on official build environment. Output is binary for distribution.

<https://gitlab.archlinux.org/archlinux/packaging>

- Servers relevant to pacman

build server
PGP server
master server
mirror server
DNS server

<https://geo.mirror.pkgbuild.com>

「マスターサーバ = 1 台の固定 URL」ではありません。単一の実体サーバ、人が SSH してファイルを置く箱ではありません。実態: Arch Linux が管理する正本リポジトリ群を Geo DNS / リバースプロキシ が束ねている。

Overview

A package management utility that tracks installed packages on a Linux system. It features - dependency support - packagegroups - install and uninstall scripts - the ability to sync your local machine with a remote repository to automatically upgrade packages. - Pacman packages are a zipped tar format. - SYNOPSIS

`pacman <operation> [options] [targets]`

entities

- sync db
 - core.db
 - extra.db
 - multilib.db
- db file
 - core.files
 - extra.files

- multilib.files

libalpm (Arch Linux Package Management): Library executed under the hood

- Since version 3.0.0, pacman has been the front-end to libalpm(3).
- You can write alternative front-ends with this.

What happens when you invoke pacman

Specifies an operation with any potential options and targets to operate on.

cache

purpose When pacman downloads packages, it saves them in a cache directory. - It allows downgrading a package without the need to retrieve the previous version through other means, such as the Arch Linux Archive. - A package that has been uninstalled can easily be reinstalled directly from the cache directory, without requiring a new download from the repository.

/var/cache/pacman/pkg

databases

- are saved for every sync DB you download from
- are not deleted even if they are removed from the configuration file pacman.conf(5).

/var/lib/pacman/sync

- 配下に、次のような DB がある。ここには、bsdtar で解凍できるアーカイブファイルが置かれている。
- これらは、bsdtar したら、大量のパッケージ名のフォルダがおいてある。
- desc ファイルがあり、以下のようなことが書かれている。

%FILENAME%

zzuf-0.15-3-x86_64.pkg.tar.zst

%NAME%

zzuf

%BASE%

zzuf

%VERSION%

0.15-3

%DESC%

Transparent application input fuzzer

%CSIZE%

65826

%ISIZE%

179092

%MD5SUM%

b6af31d617c76039d0802ad3150a0ebf

%SHA256SUM%

e07ab79aa37f4a0bf1881cd9f153749ede5cf28516359fb9bab8dec4f32e0e31

%PGPSIG%

iQIzBAABCAAdFiEEtZcfLFwQqaCMYAMPeGxj8zDXy5IFAl7L5X0ACgkQeGxj8zDXy5KT0hAAga0sDQYHLSbImzeQGhd2YNKeZJJ
daPjAiY+w61hXuVpewK7/9G6j9+aroLDLbWk0kgZWUJzSNQfyTmmsdA199Pnvowlz9pvL6JzA0DkA9wQc3TmJYNqu0zcI+juXk8
l20pCuVFhIH40NkoMwvwIV8RdQ5jhWIC4KJ3fZdM6YGgpq8DYX1k+w/
j7PfqnAZnV6MaPtL7wM0qZxWLY9pYZvZyH+uM7Rb6bTqJNxN235TKQnmghgk2Y2Q7yUmgKcKvrbko6YHT+wSc6opflsoh/
+KzuRN+I/
y4qGEiP2gPLCHReEI5TAGxddsoVnIs0tPhJT8er90+JQzEk6mGGD9I2Ss9MYpv+SRLP2L50opEX5CEXc26NvoPuQFgTxkXLo4=

%URL%

https://github.com/samhocevar/zzuf

%LICENSE%

custom:WTF

%ARCH%

x86_64

%BUILDDATE%

1590420684

%PACKAGER%

Felix Yan <felixonmars@archlinux.org>

%DEPENDS%

glibc

<operation> and [options]

リモートサーバは、ただ静的な DB ファイルがあるだけで、パッケージについて検索したり走査したりするものは、全て HTTP で取ってきた sync DB をローカルで検索してるだけ。

したがって; - ミラーが成立する (静的ファイルをコピーするだけでミラーになる) - pacman.conf の Server 行がただの URL で済む - オフラインでも -Ss/-Qu/-F が動く (結果は最後の -y 時点のスナップショット) - 逆に言うと -y を怠ると全操作が古い世界観で動く - “-Su の前に -y しろ” の理由もこれ

[CREATE] Remote DB

- -s: -s: Synchronize packages from the remote repositories, including all dependencies required to run the packages.
- -u: #### [READ] Local DB

- -Q: Read information on pacman DB
- -F: Search for information on pacman DB files ##### [UPDATE]
- -D: ##### [DELETE]
- -R:

[targets]

- A target is usually a package name, file name, URL, or a search string, which can be provided as command line arguments. Additionally, if stdin is not from a terminal and a single hyphen (-) is passed as an argument, targets will be read from stdin.

pacman -S

- -s: -s: Synchronize packages from the remote repositories, including all dependencies required to run the packages.
- You can specify:
 - the repository using a space- and/or comma-separated list of package numbers: `pacman -S testing/qt`.
 - version requirements with "`pkg-name>=X.X`".
 - groups can be specified as well. Sequential packages may be selected by specifying the first and last package numbers separated by a hyphen (-). Excluding packages is achieved by prefixing a number or range of numbers with a caret (^).

Packages that provide other packages are also handled. A selection prompt is provided if multiple packages providing foo are found. パッケージの新旧を比較するときは、以下の基準で比較する。

Alphanumeric:

`1.0a < 1.0b < 1.0beta < 1.0p < 1.0pre < 1.0rc < 1.0 < 1.0.a < 1.0.1`

Numeric:

`1 < 1.0 < 1.1 < 1.1.1 < 1.2 < 2.0 < 3.0.0`

-y Download a fresh copy of the master package databases (repo.db) from the server. ダウンロード元のサーバの URL は、`pacman.conf` で指定できる。This should typically be used each time you use `-sysupgrade` or `-u`. -yy 全部最新に見えても、強制的に PKG DB の同期を矯正する

-u, `-sysupgrade core.db extra.db` などのローカル DB と、

-uu downgrade も含んで、remote に同期する。

Additionally, version strings can have an epoch value defined that will overrule any version comparison, unless the epoch values are equal.

epoch:version-rel format.

i.e. `2:1.0-1` is always greater than `1:3.6-1`.

-c: もう使われていないパッケージのキャッシュを削除する。 -cc: 使われているものも含めて、すべてのパッケージのキャッシュを削除する。

-g, -groups Display all the members for each package group specified. If no group names are provided, all groups will be listed; pass the flag twice to view all groups and their members.

-i: Display information on a given sync database package. -ii: will also display those packages in all repositories that depend on this package.

-l List all packages in the specified repositories. Multiple repositories can be specified on the command line.

-q, -quiet

-s, -search This will search each package in the sync databases for names or descriptions that match regexp. When you include multiple search terms, only packages with descriptions matching ALL of those terms will be returned.

-U, --upgrade: Upgrade or add package(s) to the system and install the required dependencies from sync repositories.

Either a URL or file path can be specified. This is a “remove-then-add” process. See Upgrade Options below; also see Handling Config Files for an explanation on how pacman takes care of configuration files.

Options to apply to -s and -u

-w, -downloadonly Retrieve all packages from the server, but do not install/upgrade anything.

-asdeps / -asexplicit Install packages non-explicitly / explicitly;

non-explicitly: fake their install reason to be installed as a dependency > useful for makepkg and other build-from-source tools that need to install dependencies before building the package.

explicitly: fake their install reason to be explicitly installed. > useful if you want to mark a dependency as explicitly installed so it will not be removed by the -recursive remove operation.

-ignore / -ignoregroup Directs pacman to ignore upgrades of package / all packages in group, even if there is one available even if there is one available. Multiple packages / groups can be specified by separating them with a comma.

-needed Do not reinstall the targets that are already up-to-date.

-overwrite Bypass file conflict checks and overwrite conflicting files. If the package that is about to be installed contains files that are already installed and match glob, this option will cause all those files to be overwritten. Using -overwrite will not allow

overwriting a directory with a file or installing packages with conflicting files and directories. Multiple patterns can be specified by separating them with a comma. May be specified multiple times. Patterns can be negated, such that files matching them will not be overwritten, by prefixing them with an exclamation mark. Subsequent matches will override previous ones. A leading literal exclamation mark or backslash needs to be escaped.

-D: Operate on the package database.

Operates on the package database, modifies certain attributes of the installed packages in pacman's database, and check the databases for internal consistency.

-asdeps / -asexplicit Mark a package as non-explicitly / explicitly installed; in other words, set their install reason to be installed "as a dependency" / "explicitly installed".

-k, -check (available in -D and -Q) Check that the local package database is internally consistent and that all files owned by the given package(s) are present on the system. If packages are not specified or filter flags are not provided, check all installed packages. This will check all required files are present and that installed packages have the required dependencies, do not conflict and that multiple packages do not own the same file. Specifying this option twice will perform: - a check on the sync databases to ensure all specified dependencies are available - more detailed file checking including: - permissions - file sizes - modification times -> for packages that contain the needed mtree file.

-q, --quiet

-Q: Query the package database.

View installed packages, their files, and meta information about individual packages (dependencies, conflicts, install date, build date, size). Can be run against the local package database or can be used on individual package files. In the first case, if no package names are provided in the command line, all installed packages will be queried.

-c, --changelog View the ChangeLog of a package if it exists.

-d, -deps / -e, -explicit Restrict or filter output to packages installed as dependencies. This option can be combined with -t for listing real orphans packages that were installed as dependencies but are no longer required by any installed package. Restrict or filter output to explicitly installed packages. This option can be combined with -t to list explicitly installed packages that are not required by any other package.

-g, -groups Display all packages that are members of a named group. If a name is not specified, list all grouped packages.

-i, -info Display information on a given package. The -p option can be used if querying a package file instead of the local database. Passing two -info or -i flags will also display the list of backup files and their modification states.

-l, -list List all files owned by a given package. Multiple packages can be specified on the command line.

-n, -native / -m, -foreign Restrict or filter output to packages that were found / not-found in the sync database(s).

Typically packages that were downloaded manually and installed with -upgrade.

-o, -owns Search for packages that own the specified file(s). The path can be relative or absolute, and one or more files can be specified.

-p, -file Signifies that the package supplied on the command line is a file and not an entry in the database. The file will be decompressed and queried. This is useful in combination with -info and -list.

-q, -quiet

-s, -search Search each locally-installed package for names or descriptions that match regexp. When including multiple search terms, only packages with descriptions matching ALL of those terms are returned.

-t, -unrequired Restrict or filter output to print only packages neither required nor optionally required by any currently installed package. Specify this option twice to include packages which are optionally, but not directly, required by another package.

-u, -upgrades Restrict or filter output to packages that are out-of-date on the local system. Only package versions are used to find outdated packages; replacements are not checked here. This option works best if the sync database is refreshed using -Sy.

-R: Remove package(s) / group(s) from the system.

Files belonging to the specified package will be deleted, and the database will be updated. Most configuration files will be saved with a .pacsave extension unless the -nosave option is used. See Remove Options below.

-n, -nosave Instructs pacman to ignore file backup designations. Normally, when a file is removed from the system, the database is checked to see if the file should be renamed with a .pacsave extension.

```
pacman -Rc <target> # --cascade
# Removes all target packages, all packages that depend on one or more target
packages. This operation is recursive and
```

```
pacman -Ru <target> # --unneeded
```

```
# targets that are not required by any other packages must be used with care,
since it can remove many potentially needed packages to avoid breaking any
dependencies.
```

-s, -recursive

pacman -Rs

```
# Remove each target specified including all of their dependencies.
```

provided that (A) they are not required by other packages (B) they were not explicitly installed by the user This operation is recursive and analogous to a backwards -sync operation. It helps keep a clean system without orphans. If you want to omit condition (B), pass this option twice (`pacman -Rss`)

Options to apply to -s, -R and -u (transaction options)

-d, -nodeps Skips dependency version checks. Package names are still checked. Normally, pacman will always check a package's dependency fields to ensure that all dependencies are installed and there are no package conflicts in the system. Specify this option twice to skip all dependency checks.

-assume-installed <package=version> Add a virtual package "package" with version "version" to the transaction to satisfy dependencies. This allows disabling the specific dependency checks without affecting all dependency checks. To disable all dependency checking, see the **-nodeps** option.

-dbonly Adds/removes the database entry only, leaving all files in place.

-noscriptlet If an install scriptlet exists, do not execute it. Do not use this unless you know what you are doing.

-p, -print Only print the targets instead of performing the actual operation (sync, remove or upgrade). Use **-print-format** to specify how targets are displayed. The default format string is "%l", which displays URLs with -S, file names with -U, and pkgname-pkgver with -R.

-print-format Specify a printf-like format to control the output of the **-print** operation. The possible attributes are: "%a" for arch, "%b" for builddate, "%d" for description, "%e" for pkgbase, "%f" for filename, "%g" for base64 encoded PGP signature, "%h" for sha256sum, "%m" for md5sum, "%n" for pkgname, "%p" for packager, "%v" for pkgver, "%l" for location, "%r" for repository, "%s" for size, "%C" for checkdepends, "%D" for depends, "%G" for groups, "%H" for conflicts, "%L" for licenses, "%M" for madeke - pends, "%O" for optional depends, "%P" for provides and "%R" for replaces. Implies **-print**.

-nopprogressbar Do not show a progress bar when downloading files. This can be useful for scripts that call pacman and capture the output.

Test

-T, --deptest Check dependencies; this is useful in scripts such as makepkg to check installed packages. This operation will check each dependency specified and return a list of dependencies that are not currently satisfied on the system. This operation accepts no other options. Example usage: `pacman -T qt "bash>=3.2"`.

Search

-F: Query the files database. This operation allows you to look for packages owning certain files or display files owned by certain packages.

-y Download fresh package file databases (repo.files) from the server. Use twice to force a refresh even if databases are up to date.

-l List the files owned by the queried package.

-x, --regex Interpret each query as a regular expression.

-q --machinereadable Print each match in a machine readable output format. The format is repository\0pkgname\0pkgver\0pathwith \0 being the NULL character and a linefeed.

-v: Version

`pacman -V`

pacman OPTIONS

-h, --help

`pacman -Sh`

`pacman -Uh`

`pacman -Qh`

`pacman -Fh`

`pacman -Uh`

`pacman -Dh`

`pacman -Rh`

`pacman -Vh`

`pacman -h`

-b, --dbpath Specify an alternative database location (the default is /var/lib/pacman). This should not be used unless you know what you are doing. NOTE: If specified, this is an absolute path, and the root path is not automatically prepended.

-r, --root <path>

Specify an alternative installation root (default is /). This should not be used as a way to install software into /usr/local

instead of /usr. NOTE: If database path or log file are not specified on either the command line or in pacman.conf(5), their

default location will be inside this root path. NOTE: This option is not suitable for performing operations on a mounted guest

system. See `--sysroot` instead.

`-v, --verbose`
Output paths such as the Root, Conf File, DB Path, Cache Dirs, etc.

`--arch <arch>`
Specify an alternate architecture.

`--cachedir <dir>`
Specify an alternative package cache location (the default is `/var/cache/pacman/pkg`). Multiple cache directories can be specified, and they are tried in the order they are passed to pacman. NOTE: This is an absolute path, and the root path is not automatically prepended. If `DownloadUser` is set in `pacman.conf(5)`, then the specified user must have permission to access the cache directory.

`--color <when>`
Specify when to enable coloring. Valid options are `always`, `never`, or `auto`. `always` forces colors on; `never` forces colors off; and `auto` only automatically enables colors when outputting onto a tty.

`--config <file>`
Specify an alternate configuration file.

`--debug`
Display debug messages. When reporting bugs, this option is recommended to be used.

`--gpgdir <dir>`
Specify a directory of files used by GnuPG to verify package signatures (the default is `/etc/pacman.d/gnupg`). This directory should contain two files: `pubring.gpg` and `trustdb.gpg`. `pubring.gpg` holds the public keys of all packagers. `trustdb.gpg` contains a so-called trust database, which specifies that the keys are authentic and trusted. NOTE: This is an absolute path, and the root path is not automatically prepended.

`--hookdir <dir>`
Specify an alternative directory containing hook files (the default is `/etc/pacman.d/hooks`). Multiple hook directories can be specified with hooks in later directories taking precedence over hooks in earlier directories. NOTE: This is an absolute path, and the root path is not automatically prepended.

`--logfile <file>`
Specify an alternate log file. This is an absolute path, regardless of the installation root setting.

`-confirm/-noconfirm` Cancels the effects of a previous `-noconfirm`. Bypass any and all “Are you sure?” messages. It’s not a good idea to do this unless you want to run pacman from a script.

`-disable-download-timeout` Disable defaults for low speed limit and timeout on downloads. Use this if you have issues downloading files with proxy and/or security gateway.

`-sysroot` Specify an alternative system root. This path will be prepended to all other configuration directories and any repository servers beginning with `file://`. Any paths or URLs passed as targets will not be modified. This allows mounted guest systems to be properly operated on.

`-disable-sandbox` Completely disables the sandbox applied to the process of downloading files on Linux systems. Equivalent to jointly specifying `-disable-sandbox-filesystem` and `-disable-sandbox-syscalls`.

`-disable-sandbox-filesystem` Disable the filesystem restrictions part of the sandbox applied to the process downloading files on Linux systems. Useful if experiencing Landlock related failures while downloading files when running a Linux kernel that does not support this feature.

`-disable-sandbox-syscalls` Disable the syscall filtering part of the sandbox applied to the process downloading files on Linux systems. Useful if experiencing seccomp related failures while downloading files when running a Linux kernel that does not support this feature.

paru

alpm-hooks

libalpm

pacman-contrib

makepkg

pacman.conf

- `cache` If you use a network shared cache, see the `CleanMethod` option in `pacman.conf(5)`.

Pacman uses the same logic as rpm to determine action against files that are designated to be backed up. During an upgrade, three MD5 hashes are used for each backup file to determine the required action: one for the original file installed, one for the new file that is about to be installed, and one for the actual file existing on the file system. After comparing these three hashes, the following scenarios can result:

original=X, current=X, new=X All three files are the same, so overwrites are not an issue. Install the new file.

original=X, current=X, new=Y The current file is the same as the original, but the new one differs. Since the user did not ever modify the file, and the new one may contain improvements or bug fixes, install the new file.

original=X, current=Y, new=X Both package versions contain the exact same file, but the one on the file system has been modified. Leave the current file in place.

original=X, current=Y, new=Y The new file is identical to the current file. Install the new file.

original=X, current=Y, new=Z All three files are different, so install the new file with a .pacnew extension and warn the user. The user must then manually merge any necessary changes into the original file.

original=NULL, current=Y, new=Z The package was not previously installed, and the file already exists on the file system. Install the new file with a .pacnew extension and warn the user. The user must then manually merge any necessary changes into the original file.